

Practica 1 - Fuerza Bruta y Backtracking

Analisis y Diseno de Algoritmos

Integrantes

Samuel Serna
Nicolas Ortiz

Fecha: 30 de agosto de 2026

2. Introduccion

Esta practica estudia experimentalmente dos paradigmas algoritmicos: Fuerza Bruta y Backtracking. Ambos trabajan sobre espacios de busqueda asociados a contraseñas sintéticas, pero resuelven problemas diferentes.

En Fuerza Bruta se analiza el costo de encontrar una contraseña mediante enumeración sistemática y verificación por hash SHA-256, comparandola también con una búsqueda por diccionario. En Backtracking se construyen contraseñas que satisfacen una política de composición, usando poda para evitar estados que no pueden conducir a una solución válida.

El objetivo es relacionar el análisis teórico de complejidad con mediciones de tiempo, candidatos o estados explorados y reducción del espacio de búsqueda.

3. Contexto del problema

Fuerza Bruta

El problema consiste en recuperar una contraseña sintética a partir de su hash SHA-256. Como el hash no permite recuperar directamente el texto original, se generan candidatos, se calcula el hash de cada uno y se compara con el hash objetivo. La Fuerza Bruta puede cubrir todo el espacio definido por alfabeto y longitud; el diccionario prueba solamente 500 candidatos y por eso es rápido, aunque puede no contener el objetivo.

Backtracking

Se generan contraseñas sujetas a mínimos de minúsculas, mayúsculas, dígitos y símbolos, además de la prohibición de caracteres idénticos consecutivos. En vez de generar todas las cadenas y validarlas al final, se construyen de forma incremental y se descartan anticipadamente prefijos inviables.

4. Fundamentacion teorica

Fuerza Bruta

Si el alfabeto tiene tamaño b y la longitud es n , existen b^n candidatos. El crecimiento es exponencial. Para cada candidato se calcula SHA-256 y se compara con el objetivo. Una búsqueda por diccionario con d entradas tiene costo lineal en d , pero no garantiza encontrar la contraseña.

Backtracking

Backtracking recorre un árbol de decisiones en profundidad. Cada nodo es una solución parcial y cada arista agrega un carácter. La poda elimina ramas cuando, con las posiciones restantes, ya no es posible cumplir los requisitos. Si ninguna restricción permite podar, el comportamiento se aproxima a la enumeración exhaustiva.

5. Modelamiento

Fuerza Bruta

- Hash SHA-256 objetivo.
- Alfabeto A1: 26 letras minúsculas.
- Alfabeto A2: 26 letras minúsculas + 10 dígitos (36 símbolos).
- Longitud mínima y máxima.
- Diccionario de 500 candidatos.

Las instancias privadas se generan con los apellidos normalizados y ordenados: ortiz + serna. La suma ASCII produce la semilla 1105.

Backtracking

- Estado: prefijo, conteo de minúsculas, mayúsculas, dígitos y símbolos.
- Política del equipo: $n=8$, $\text{minLower}=3$, $\text{minUpper}=2$, $\text{minDigit}=2$, $\text{minSymbol}=1$ y sin repetidos consecutivos.
- Alfabeto implementado: 26 minúsculas + 26 mayúsculas + 10 dígitos + 5 símbolos (!@#\$%) = 67 caracteres.

El enunciado menciona 69 símbolos, pero los conjuntos enumerados explícitamente suman 67; se implementaron los caracteres especificados.

6. Diseño algoritmico

Fuerza Bruta

Se enumeran los candidatos del espacio definido. Para cada uno se calcula SHA-256 y se compara con el hash objetivo; si coincide, la búsqueda termina. El modo diccionario realiza el mismo proceso sobre las 500 palabras del archivo. Se incluyó generación reproducible de instancias y ejecución por lotes desde CSV.

Backtracking

Se agrega un caracter a la vez. Antes de expandir se calcula cuantos caracteres de cada categoria faltan y se compara con las posiciones disponibles. Si no alcanzan, el estado se poda. Tambien se rechaza inmediatamente una repeticion consecutiva cuando la politica la prohíbe. Se implemento una version sin poda para comparacion.

7. Pseudocodigo

Fuerza Bruta

```
FUERZA_BRUTA(hash, alfabeto, n)
  PARA cada candidato de longitud n
    SI SHA256(candidato) = hash: RETORNAR candidato
  RETORNAR no encontrado

DICCIONARIO(hash, archivo)
  PARA cada palabra del archivo
    SI SHA256(palabra) = hash: RETORNAR palabra
  RETORNAR no encontrado
```

Backtracking con poda

```
BACKTRACKING(estado)
  contar nodo visitado
  SI estado no factible: contar poda y RETORNAR
  SI longitud(prefijo)=n: validar y RETORNAR
  PARA cada caracter c
    SI repite consecutivamente: contar poda y CONTINUAR
    agregar c
    SI nuevo estado factible: BACKTRACKING(estado)
  SINO: contar poda
  retirar c
```

Enumeracion sin poda

La version sin poda expande todos los caracteres hasta profundidad n y valida unicamente al llegar a una cadena completa.

8. Implementacion

Fuerza Bruta

- src/fb_core.cpp y .hpp: generacion exhaustiva y hash.
- src/fb_dictionary.cpp y .hpp: busqueda en diccionario.
- src/fb_instancias.cpp y .hpp: semilla e instancias privadas.
- src/third_party/picosha2.h: SHA-256.
- tests/test_fb.cpp: pruebas automaticas.

Backtracking

- src/bt_backtracking.cpp y .hpp: algoritmo con y sin poda.
- src/main.cpp: interfaz integrada.
- tests/test_backtracking.cpp: pruebas automaticas.
- Metricas: nodos visitados, podados, soluciones, tiempo y estado de interrupcion.
- Limite configurable de nodos para ejecuciones grandes.

9. Analisis de complejidad

Fuerza Bruta

Para longitud fija, el tiempo es $O(b^n)$. Para un rango $[min, max]$, se exploran $b^{min} + \dots + b^{max}$ candidatos. El diccionario con d entradas tiene $O(d)$; aqui $d=500$.

Backtracking

El peor caso sigue siendo $O(b^n)$. El arbol completo contiene $1+b+b^2+\dots+b^n$ nodos. La pila de recursion tiene profundidad maxima n, por lo que el espacio auxiliar es $O(n)$ cuando las soluciones solo se contabilizan.

10. Casos de prueba

Fuerza Bruta

Se evaluaron 13 configuraciones: A1 con $n=3,4,5,6$; A2 con $n=3,4,5$; la referencia abc12 con A2 y $n=5$; y cinco instancias privadas de la semilla 1105. Las pruebas verificaron SHA-256, busqueda exhaustiva, diccionario, semilla e instancias. Todas pasaron.

Backtracking

Se usaron la política del equipo con n=6,8,10; política relajada n=8; sin restricciones n=6; referencia; y una instancia pequeña con alfabeto abAB12!, n=4 y mínimo uno por categoría. En esta última, ambas versiones encontraron 192 soluciones. Todas las pruebas automáticas pasaron.

11. Experimentación

Fuerza Bruta

Config.	Espacio teórico	Candidatos	Tiempo
A1 n=3	17,576	17,576	16.357 ms
A1 n=4	456,976	456,976	483.164 ms
A1 n=5	11,881,376	11,881,376	11,296.6 ms
A1 n=6	308,915,776	308,915,776	323,710 ms
A2 n=3	46,656	46,656	43.040 ms
A2 n=4	1,679,616	1,679,616	1,577.59 ms
A2 n=5	60,466,176	60,466,176	57,886.8 ms

A1 n=6 tarda aproximadamente 323.7 s (5.4 min), punto donde el costo ya comienza a ser poco manejable para experimentación local.

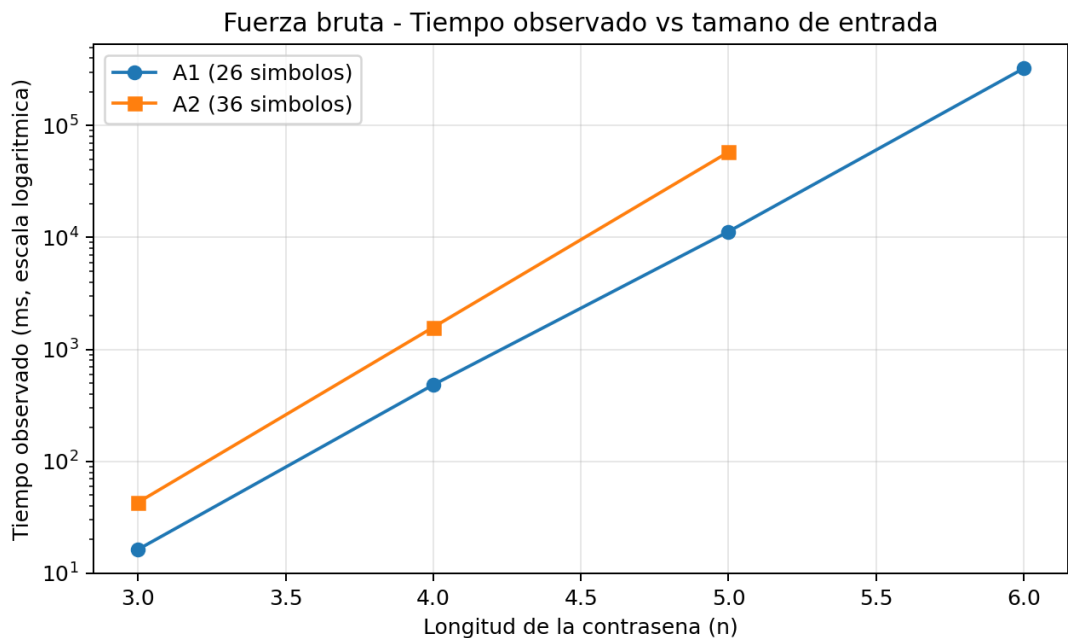


Figura 1. Tiempo de Fuerza Bruta frente a la longitud.

Backtracking

Métrica	Con poda	Sin poda
Nodos visitados	356	2801
Soluciones	192	192
Tiempo última ejecución	0.032 ms	0.085 ms
Reducción de nodos	87.290%	-

Las ejecuciones grandes se marcaron como INTERRUMPIDO POR LIMITE cuando alcanzaron el máximo de nodos configurado.

12. Resultados

Fuerza Bruta

La Fuerza Bruta encontró 13 de 13 objetivos (100% de éxito). La referencia abc12 requirió 50,249 candidatos y 49.1765 ms. El diccionario evaluó 500 candidatos por instancia y encontró 0 de 13 objetivos (0% de éxito), con tiempos de aproximadamente 0.46 a 0.68 ms.

Instancia	Alfabeto	n	Candidatos FB	Tiempo
equipo_1	A1	4	244,917	226.558 ms
equipo_2	A2	4	833,017	791.850 ms
equipo_3	A1	5	11,682,276	11,049.9 ms
equipo_4	A2	5	4,262,007	4,012.32 ms
equipo_5	A1	6	130,784,415	125,835 ms

Backtracking

- equipo_n6: 1 nodo visitado, 1 podado, 0 soluciones, COMPLETADO; la suma de minimos es 8 y $n=6$.
- equipo_n8, limite 10M: 10,000,000 visitados; 52,628,655 podados; 9,065,240 soluciones parciales; 423.289 ms.
- equipo_n10, limite 10M: 10,000,000 visitados; 52,628,656 podados; 9,065,238 soluciones parciales; 431.392 ms.
- relajada_n8, limite 10M: 151,517 podados; 9,848,479 soluciones parciales; 130.724 ms.
- sin_restricciones_n6, limite 10M: 0 podados; 9,850,742 soluciones parciales; 102.420 ms.

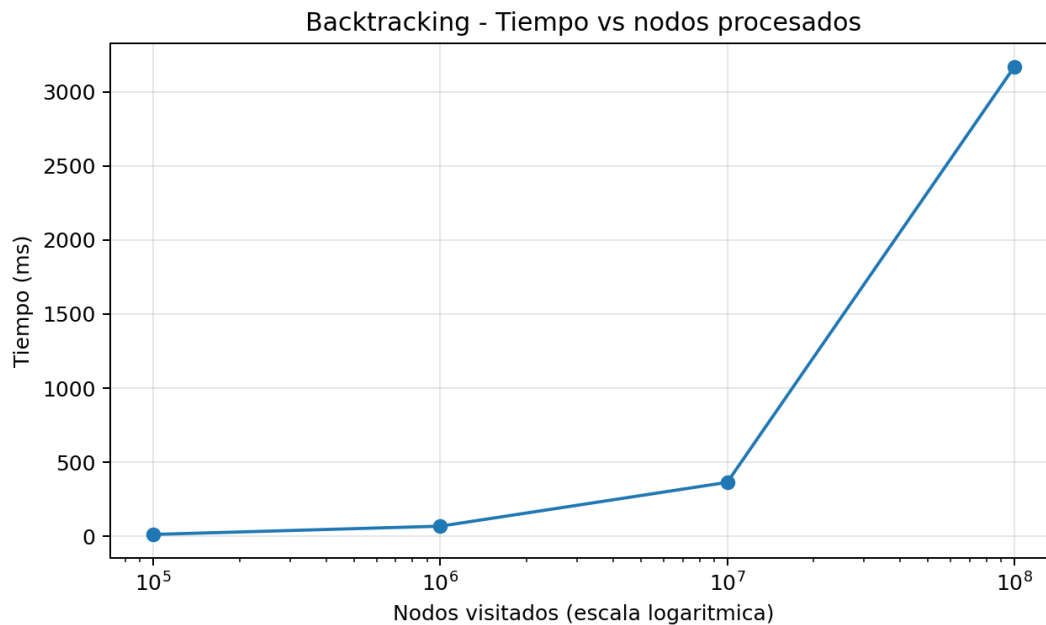


Figura 2. Tiempo de Backtracking frente a nodos procesados.

13. Analisis de resultados

Fuerza Bruta

Los datos confirman el crecimiento exponencial. En A1, el espacio pasa de 17,576 candidatos en $n=3$ a 308,915,776 en $n=6$, y el tiempo crece de 16.4 ms a 323.7 s. En A2, $n=5$ ya implica 60,466,176 candidatos y 57.9 s. A2 con $n=6$ tendría $36^6 = 2,176,782,336$ candidatos, por lo que el costo esperado sería mucho mayor.

El diccionario es mucho mas rapido porque limita el espacio a 500 candidatos, pero su tasa de exito fue 0% porque ninguna contrasena objetivo estaba incluida. Esto evidencia el intercambio entre cobertura y costo.

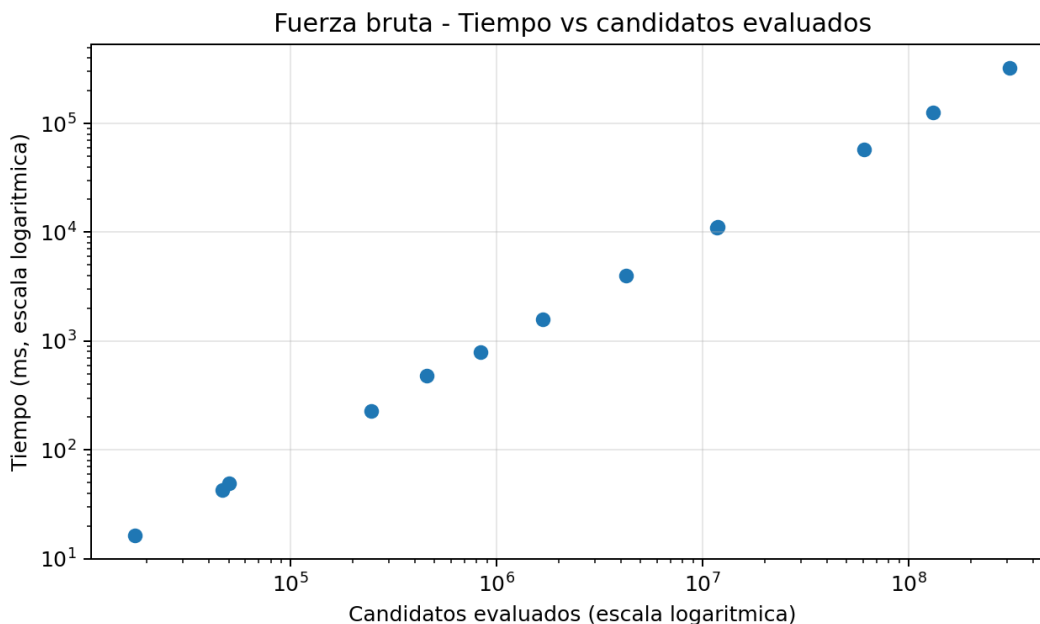


Figura 3. Relacion entre candidatos evaluados y tiempo de Fuerza Bruta.

Backtracking

La poda es efectiva cuando las restricciones permiten detectar inviabilidad. equipo_n6 se resuelve en la raíz. Sin restricciones, no se poda ningún nodo. La instancia pequeña confirma corrección: ambas versiones obtienen 192 soluciones, mientras la poda reduce los nodos de 2801 a 356 (87.290%). Las soluciones de ejecuciones limitadas se reportan expresamente como parciales.

14. Comparacion algoritmica

Fuerza Bruta pura vs. diccionario

Fuerza Bruta obtuvo 100% de éxito en las 13 instancias y el diccionario 0%. El diccionario tarda menos de 1 ms al evaluar solo 500 palabras, pero depende de que el objetivo esté en ese conjunto. Fuerza Bruta ofrece cobertura completa dentro del alfabeto y longitud definidos, a costa de crecimiento exponencial.

Backtracking con poda vs. sin poda

En la prueba completa, la poda visita 356 nodos frente a 2801 sin poda y conserva las mismas 192 soluciones. En equipo_n6 la versión con poda termina en la raíz, mientras que la versión sin poda alcanza 1,000,000 de nodos sin completar el árbol. Esa segunda comparación es parcial y se usa solo para evidenciar costo.

15. Conclusiones

Los experimentos comprobaron el crecimiento exponencial de los espacios de búsqueda. En Fuerza Bruta, incrementar n o ampliar el alfabeto eleva rápidamente candidatos y tiempo; A1 $n=6$ ya requirió aproximadamente 5.4 minutos.

El diccionario fue mucho más rápido, pero no encontró ninguna contraseña, mostrando que su eficiencia depende de la cobertura de candidatos. Backtracking redujo drásticamente el trabajo cuando las restricciones permitieron identificar prefijos inviables, pero sin restricciones no produjo poda.

La comparación completa de Backtracking demostró que la poda mantiene la corrección y reduce 87.290% los nodos visitados. En conjunto, los resultados muestran que la complejidad teórica, la estructura del espacio y la calidad de las restricciones determinan el comportamiento práctico.

16. Referencias

- Material del curso de Analisis y Diseno de Algoritmos.
- Enunciado de la Practica 1 - Fuerza Bruta y Backtracking.
- PicoSHA2, biblioteca SHA-256 incluida en src/third_party/picosha2.h y su licencia.
- Biblioteca estandar de C++17, incluyendo std::chrono para medicion de tiempos.

17. Uso de herramientas de IA

Herramienta: OpenAI ChatGPT. Fecha de uso: agosto de 2026.

- Aclaracion de conceptos de Backtracking y poda.
- Revision de errores de compilacion y ejecucion.
- Apoyo para organizacion y documentacion reproducible.
- Revision de interpretacion de resultados experimentales.
- Apoyo en la integracion final de Fuerza Bruta y Backtracking y en la redaccion del informe.

El codigo integrado fue verificado mediante compilacion, pruebas automaticas y ejecuciones experimentales. No se atribuyen a Nicolas Ortiz herramientas de IA adicionales no declaradas por el.

18. Contribucion individual de los integrantes

Samuel Serna

Responsable principalmente del modulo de Backtracking: modelamiento de politica y estados, implementacion con poda y sin poda, pruebas, limite de nodos, configuraciones experimentales, metricas, comparaciones, graficas y documentacion. Tambien participo en la integracion final y verificacion conjunta.

Nicolas Ortiz

Responsable principalmente del modulo de Fuerza Bruta: busqueda exhaustiva, diccionario, integracion SHA-256 con PicoSHA2, semilla e instancias privadas, pruebas, experimentacion, registro de resultados, graficas y documentacion del modulo.